

Software Requirements Specification

For ECE1140 Train Management System

Version 1.0

Prepared by Team 3

University of Pittsburgh

2026-08-28

Table of Contents

- [1. Introduction](#)
 - [1.1 Document Purpose](#)
 - [1.2 Product Scope](#)
 - [1.3 Definitions, Acronyms, and Abbreviations](#)
 - [1.4 References](#)
 - [1.5 Document Overview](#)
- [2. Product Overview](#)
 - [2.1 Product Perspective](#)
 - [2.2 Product Functions](#)
 - [2.3 Product Constraints](#)
 - [2.4 User Characteristics](#)
 - [2.5 Assumptions and Dependencies](#)
 - [2.6 Apportioning of Requirements](#)
- [3. Requirements](#)
 - [3.1 External Interfaces](#)
 - [3.2 Functional](#)
 - [3.3 Quality of Service](#)
 - [3.4 Compliance](#)
 - [3.5 Design and Implementation](#)
 - [3.6 AI/ML](#)
- [4. Verification](#)
- [5. Appendixes](#)

Revision History

Name	Date	Reason For Changes	Version
Team 3	2026-08-28	Initial document draft	1.0

1. Introduction

This SRS defines the software requirements for the ECE1140 Train Management System, a simulated rail network developed as a term-long group project. It covers functional behavior, quality attributes, and verification criteria for the system.

1.1 Document Purpose

This document defines what the ECE1140 Train Management System must do. It is intended for the development team, QA, and the course instructor, and serves as the baseline for design, testing, and grading throughout the project lifecycle.

1.2 Product Scope

The ECE1140 Train Management System is a Python-based application that provides a train management service with a graphical user interface. It allows operators to dispatch and monitor trains on a simulated rail network.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
API	Application Programming Interface
Authority	The maximum distance a train is permitted to travel before stopping
Block	A discrete section of track that can be occupied by at most one train
CTC	Centralized Traffic Control — the operator dispatching interface
SRS	Software Requirements Specification
UI	User Interface
UML	Unified Modeling Language

1.4 References

#	Title	Author	Date	Type
1	ECE1140 Course Project Description	University of Pittsburgh	2026-08	Normative
2	IEEE Std 830-1998: Recommended Practice for SRS	IEEE	1998	Informative

1.5 Document Overview

Section 2 provides product context, functions, and constraints. Section 3 contains all verifiable requirements. Section 4 maps each requirement to a verification method. Section 5 contains supporting appendixes. Requirements marked "shall" are mandatory; "should" is recommended; "may" is optional.

2. Product Overview

2.1 Product Perspective

The ECE1140 Train Management System is a new, standalone Python application developed for the ECE1140 course project. It is not a replacement for any existing system. The system simulates a complete commuter rail network and exposes operator, driver, and maintenance interfaces through a graphical UI.

2.2 Product Functions

The system provides a train management service through a graphical UI, including:

- Dispatching and monitoring trains on a simulated network
- Displaying live track and train status to the operator
- Enforcing safe train separation and speed limits
- Simulating train motion, station stops, and failure conditions

2.3 Product Constraints

- The system shall be implemented in Python 3.10 or later
- The system shall run on any standard modern PC (e.g., 8 GB RAM, 2 GHz CPU)

2.4 User Characteristics

User Class	Description	Expertise
CTC Operator	Dispatches trains and monitors the network	No programming knowledge required
Train Driver	Controls a single train in manual mode	Basic operational training
Maintenance Engineer	Injects failures for testing	Technical; understands system internals
Course Instructor	Evaluates the system against the grading rubric	Full technical expertise

2.5 Assumptions and Dependencies

#	Assumption / Dependency	Impact if False
A1	Course-provided track layout CSV files are correct and complete	Track simulation would require rework
A2	Lab machines have Python 3.10+ available	Additional setup time required
A3	Module interface contracts are agreed by end of Sprint 1	Late-stage integration failures
A4	Sprint end dates do not change after Week 2	Requirement phasing may need revision

2.6 Apportioning of Requirements

Functional Area	Sprint 1	Sprint 2	Sprint 3	Sprint 4
Dispatching & Scheduling	Basic UI shell	Schedule entry	Auto-dispatch	Full dispatch
Safety & Control	Interface contracts	Authority logic	Switch/signal control	Full safety demo
Train & Track Simulation	Layout file loading	Physics/occupancy	Failure injection	Complete simulation
Station Operations	—	Door logic	Passenger counts	Full station behavior

3. Requirements

3.1 External Interfaces

3.1.1 User Interfaces

The system shall provide a graphical UI for operators to dispatch and monitor trains, for drivers to view train status and control their train in manual mode, and for maintenance engineers to inject and clear simulated failures.

3.1.2 Hardware Interfaces

The system requires no physical hardware beyond a standard keyboard, mouse, and monitor. All train and track behavior is simulated in software.

3.1.3 Software Interfaces

The system shall load track layout data from the course-provided CSV files at startup. All internal components shall share a single simulation clock supporting real-time and fast-forward operation.

3.2 Functional

The system shall provide the following capabilities:

- **Train Dispatching:** Operators can dispatch trains manually or from a pre-loaded schedule.
- **Live Monitoring:** The UI displays the current state of all trains, track blocks, switches, and signals.
- **Safe Separation:** The system tracks block occupancy and limits each train's movement to prevent collisions.
- **Speed and Switch Control:** The system commands train speeds within block limits and positions switches according to each train's route.
- **Emergency Braking:** The system brings a train to a stop when its authority is exhausted, a failure is detected, or the driver requests it.
- **Train Physics:** The system simulates train velocity and position based on power, braking, and track grade.
- **Station Operations:** The system manages door open/close cycles and passenger counts at each station stop.
- **Failure Handling:** The system detects injected failures and responds safely without requiring a full restart.

3.3 Quality of Service

3.3.1 Performance

The system shall run smoothly in real-time and support fast-forward simulation without loss of correctness.

3.3.2 Security

The system shall validate all user inputs and reject invalid entries without altering simulation state.

3.3.3 Reliability

The system shall operate without crashes or unhandled errors during normal simulation runs.

3.3.4 Availability

The system shall start up and be ready for use within a reasonable time after launch.

3.3.5 Observability

The system shall produce a log of significant simulation events for debugging and review.

3.4 Compliance

- The team shall produce UML class diagrams, sequence diagrams, and a use case diagram as required by the course rubric.
- All source code and documentation shall be maintained in the course Git repository with commits linked to issue tracker tickets.
- Each team member shall contribute code commits in every sprint.

3.5 Design and Implementation

3.5.1 Installation

The system shall be launchable with `python main.py` after running `pip install -r requirements.txt` with no additional configuration.

3.5.2 Build and Delivery

The repository shall include a `requirements.txt` with pinned package versions so the system installs and runs consistently on any compatible machine.

3.5.3 Distribution

The system shall run on a single machine with no external services or network connections required.

3.5.4 Maintainability

Software modules shall interact through defined interfaces rather than accessing each other's internal state directly.

3.5.5 Reusability

Common utilities such as the simulation clock and event logger shall be implemented as standalone modules usable by any part of the system.

3.5.6 Portability

The system shall use platform-neutral file path handling so it runs correctly regardless of the developer's operating system.

3.5.7 Cost

No financial cost targets apply. All required tools (Python, Git, IDE) are free.

3.5.8 Deadline

Milestone	Date	Deliverables
Sprint 1	2026-09-25	Module interfaces, track loader, dispatch UI shell, POC demo
Sprint 2	2026-10-16	Train dynamics, occupancy, authority, basic scheduling
Sprint 3	2026-11-06	Full safety logic, switch/signal control, failure injection
Final Demo	2026-11-20	Complete integrated system; public presentation
Documentation	2026-11-25	UML, SRS, and test reports submitted

3.5.9 Proof of Concept

A Sprint 1 POC shall demonstrate a simulated train moving between track blocks under authority control with a basic dispatch UI, validating the core data flow before full implementation begins.

3.5.10 Change Management

Requirements changes after Sprint 1 require full team agreement and are recorded in the Revision History table.

3.6 AI/ML

Not applicable. All logic is deterministic and rule-based. No machine learning is used in v1.0.

4. Verification

Area	Verification Method
User interfaces (map, dispatch form, driver display, failure UI)	Demonstration
Functional behavior (dispatch, scheduling, occupancy, authority, speed, switching, braking, physics, doors, failures)	Test + Demonstration
Performance and reliability	Test
Input validation	Test
Event logging	Inspection
UML documentation, version control, team contributions	Inspection
Installation and single-machine deployment	Demonstration

Area	Verification Method
Module independence and shared utilities	Inspection

5. Appendixes

Appendix A — Track Layout File Format

Track layout files are CSV files provided by the course instructor. Columns include: Block Number, Block Length (m), Grade (%), Speed Limit (km/h), Infrastructure type, Station Name, and Door Side.

Appendix B — Safe Braking Distance Formula

$$d = v^2 / (2 \times a)$$

Where d is stopping distance (m), v is current speed (m/s), and a is deceleration (m/s²). Used to verify emergency braking behavior in REQ-FUNC-007.